Project: Expression Calculator in C (Basic Calculator II)

1. Project Goal (What you're building)

Design and implement a **string-based arithmetic expression calculator** in C.

Your calculator will:

- Read a **string expression** containing **non-negative integers**, `+`, `-`, `*`, `/`, and spaces
- Respect **operator precedence** (multiplication and division before addition and subtraction)
- Perform **integer division that truncates toward zero** (e.g., `7/2 = 3`, `-7/2 = -3`)
- Return the final result as a 32-bit signed integer

This project mirrors the requirements of **LeetCode 227 – Basic Calculator II**, but you'll build a complete **command-line tool** with your own parser and evaluator in C (no `eval`-like calls, no external parsing libraries).

---

2. Learning Objectives

By the end of this project, you should be able to:

1. Parse and evaluate arithmetic expressions given as **strings**.
2. Implement **operator precedence** and **left-to-right associativity** for `+`, `-`, `*`, `/`.
3. Use **stacks or equivalent state variables** to manage intermediate results.
4. Work with **C strings** (`char *`), including:
    - Skipping spaces,
    - Building multi-digit integers,
    - Handling end-of-string safely.
5. Design a small C program with:
    - A clear **API** (e.g., `int calculate(const char *s);`),
    - Separation of **parsing** and **evaluation** logic,
    - Basic **unit tests** or test harness.

---

3. Functional Requirements

Your C calculator must support at least the following:

3.1 Expression Format

- The input is a **single line string** representing a mathematical expression.
- The string may contain:
  - Non-negative integers (e.g., `0`, `12`, `345`)
  - Operators: `+`, `-`, `*`, `/`
  - Arbitrary spaces (e.g., `" 3 + 2 * 2 "` is valid)
- **No parentheses** are required or supported for this project.
- The expression is assumed to be **syntactically valid** for the core requirements.

3.2 Semantics

Evaluate the expression with the following rules:

- Multiplication and division have **higher precedence** than addition and subtraction.
- Operators of the same precedence are evaluated **left to right**.
- Integer division **truncates toward zero** (C's standard integer division behavior).
- All intermediate and final results will fit in a **32-bit signed integer** (type `int`).

3.3 Interface

Your program should:

1. Accept an expression string **from standard input**:
   - Option A: Read a single line and evaluate it once.
   - Option B (recommended): Provide a **REPL-like loop**:
     - Prompt with something like `>`
     - Read an expression
     - Print the result
     - Repeat until EOF or a special command (e.g., `quit`).
2. Print the result as a single integer followed by a newline.

Example behaviors (not exhaustive):

- Input: `3+2*2` → Output: `7`
- Input: `3/2` → Output: `1`
- Input: `3+5 / 2` → Output: `5`

---

4. Crucial Steps / Milestones

You can structure your work into the following milestones.

Step 1: Define the Core API

Define and implement at least one core function in a header file, e.g.:

```c
int calculate(const char *s);
```

This function should:

- Take a null-terminated C string `s`,
- Return the evaluated integer result.

Your `main()` should mostly:

- Read input,
- Call `calculate`,
- Print the result.

This makes it easy to unit-test `calculate` independently.

Step 2: Tokenization & Parsing Strategy

Decide how to process the string.

Recommended single-pass stack-based approach

Maintain:

```c
long current_number = 0;
char last_op = '+';
```

A stack (e.g., dynamic array or fixed-size array) of integers for partial results. Algorithm sketch: Iterate over the string character by character. For each digit, update

`current_number = current_number * 10 + (digit value).`

Ignore spaces.

When you encounter an operator or reach the end of the string: Apply the previous operator (last_op) to current_number and the stack: If last_op == ‘+’: push current_number If last_op == ‘-’: push -current_number If last_op == ‘’: pop the top t, push t current_number If last_op ==’/’: pop the top t, push t / current_number (truncating toward zero) Set last_op to the current operator. Reset current_number to 0. At the end, sum everything in the stack to get the final result. Alternative approaches (allowed but more complex): Two-pass evaluation (first *//, then +/-), Building an explicit token list and then evaluating.

Step 3: Implement Operator Precedence Correctly

Show in your design and code how 3+2*2 becomes 7 and not 10.

You are encouraged to write out a small example in your report or comments:

Char Action Stack after ‘3’ build number 3 [] ‘+’ apply ‘+’ to 3 (push) [3] ‘2’ build number 2 [3] ’*’ apply ‘+’ to 2 (push) [3, 2] ‘2’ build number 2 [3, 2] end apply ’’ to 2 (22=4) [3, 4] Final result: sum of stack [3, 4] = 7.

This is a good place to explain invariant reasoning: at any point in the scan, what does the stack represent?

Step 4: Implement the Stack & Supporting Utilities

In C, there are no built-in stacks, so you must implement your own. Options:

- Use a fixed-size array with a constant MAX_STACK_SIZE, and an index top.
- Implement a dynamic vector-like structure using malloc/realloc.
- Provide at least:
  - void push(int value);
  - int pop(void);
  - int is_empty(void);
- Include basic defensive checks:

If you exceed stack capacity, handle it gracefully (error message, exit, or expansion).

Avoid undefined behavior (no out-of-bounds array writes).

Step 5: Handle Edge Cases and Spaces

Make sure to correctly handle:

- Expressions with many spaces: ” 12 + 3 * 4 “.
- Multi-digit numbers: “123*45”.
- Long-but-valid inputs.
- Division by zero should not occur if you assume valid expressions, but you may still assert or check for it defensively.

1. Hints and Tips

- Do not use eval-like functions or third-party expression libraries. Your goal is to implement the parsing/evaluation logic yourself.
- Integer division in C:
  - a / b on int types truncates toward zero in modern C standards, which matches the requirement. Still, test negative cases like -7/2 to confirm your toolchain behavior.
- Be careful with types:
  - You may use long or long long internally if you worry about intermediate overflow, then cast to int at the end.
- Keep your code modular:
  - `calculate()` for evaluation,
  - `main()` for I/O,
  - optional helpers for stack operations and debugging.
- Add assert-based unit tests for `calculate()`:

```
calculate("3+2*2") == 7
calculate(" 3/2 ") == 1
calculate(" 3+5 / 2 ") == 5
```

- Use comments and diagrams to explain:
  - How the stack changes over time,
  - How last_op controls when multiplication/division happen immediately.

6. Deliverables

- C Source Code
- `calculator.c` containing:

```
int calculate(const char *s);
int main(void);
```

If you split into multiple files (e.g., stack.c, stack.h, calculator.h), include a simple Makefile.

- Short Design Report (1–3 pages) – Bonus points Explain:
  - The parsing strategy (single-pass with stack, or alternative),
  - How you implement operator precedence,
  - Stack design (array vs dynamic),
  - How you handle spaces and multi-digit numbers.
  - Include at least one worked example showing the evolution of:
    - current_number,
    - last_op,
    - stack contents.
- Design Artifacts (Bonus) Diagrams or sketches such as:
  - A state diagram of your main parsing loop,
  - A stack evolution table for one non-trivial expression.
  - Test Plan / Test Cases (Optional/Bonus)
  - A short text file listing: Input expressions and expected outputs, Coverage of:
  - Many spaces,
  - Large numbers,
  - Chains of operations (e.g., `2*3*4/2+5-6/3`).

7. Grading Criteria (Suggested Rubric)

You can adjust the weights, but here’s a clear structure:

Correctness (40%) For a suite of fixed tests: - Correct handling of operator precedence, - Correct handling of spaces, - Correct integer division semantics, - Robust handling of long expressions.

Code Quality & Organization (25%) - Clear function decomposition (calculate, stack helpers, etc.), - Well-chosen variable names, - Sufficient comments explaining logic, - No obvious memory errors (no out-of-bounds writes, no use-after-free).

Efficiency (15%) - Time complexity: O(n) over input length, where n is strlen(s). - Reasonable memory usage (stack size linear in number of operators/terms).

Understanding & Explanation (10%) Ability to explain: - Why the stack-based method enforces precedence, - What the stack values represent at any moment, - How the algorithm behaves on example inputs.

Bonus (up to +10%) - Design report and diagrams,

- Additional features:**
  - REPL with a quit command,
  - Basic error reporting for malformed expressions (even though the base spec assumes valid ones),
  - Support for unary plus/minus at the beginning of the expression (e.g., -3+2*4).